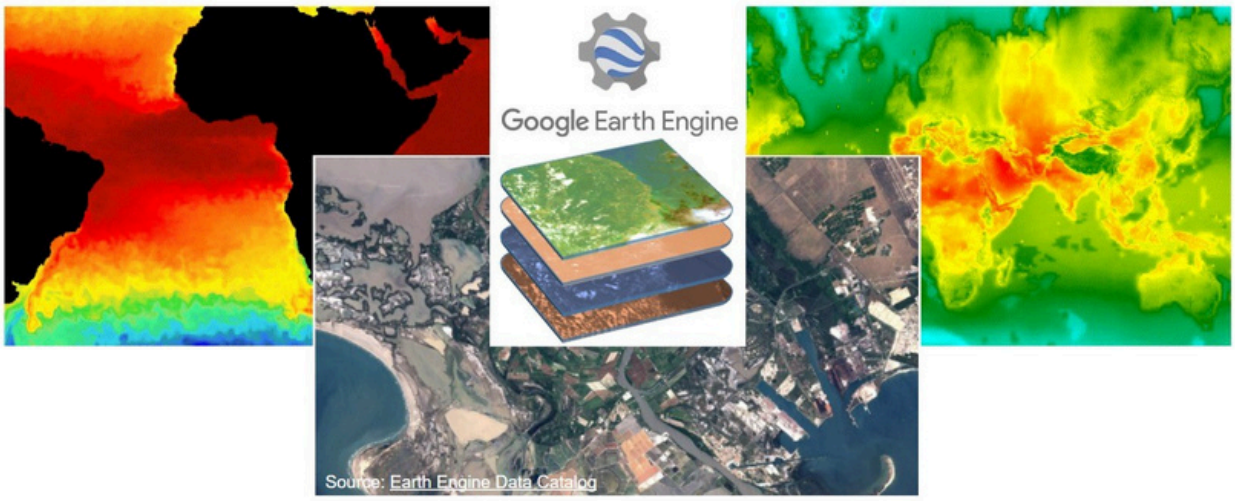


Introduction to Earth Engine

External Links and Resources

this is a great complete resource

1. **Earth Engine Guide:**
   https://developers.google.com/earth-engine/guides

2.Asking questions via GIS Stack Exchange
https://gis.stackexchange.com/questions/tagged/google-earth-engine

3.Earth Engine community and developer resources
https://developers.google.com/earth-engine/tutorials

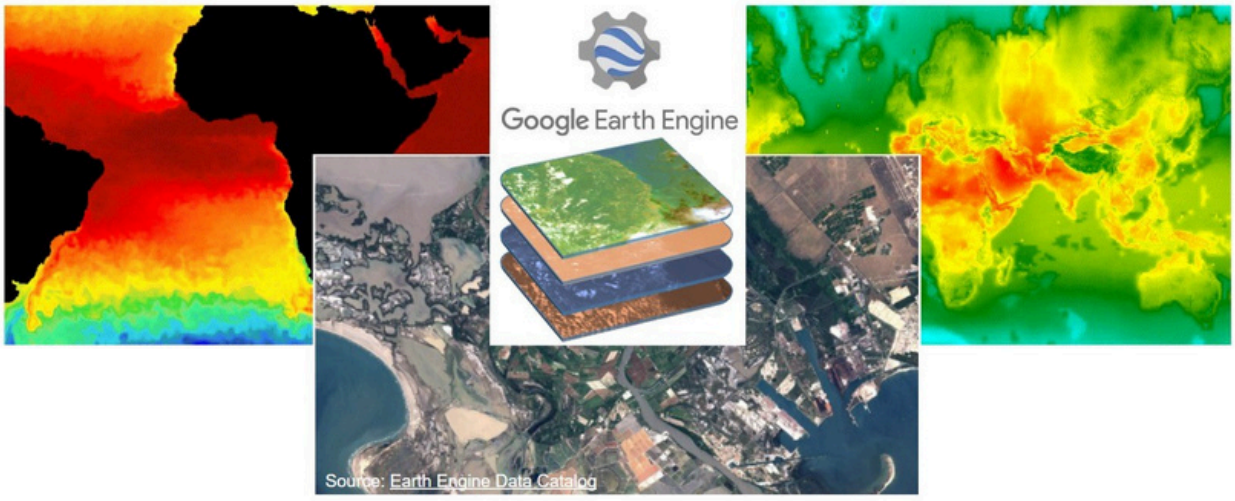
4.Python Installation
https://developers.google.com/earth-engine/guides/python_install

5.Joining Earth engine Google developer group
https://groups.google.com/g/google-earth-engine-developers

6.Stay on top of what happening
Earth Engine on Medium :https://medium.com/google-earth
Earth Engine on Twitter:https://twitter.com/googleearth

Lab#0 Preface

Registering an account



Introduction to Earth Engine

What is Google Earth Engine and what can we do with it?

Google Earth Engine (GEE) is a web-based computing platform designed to run earth science data analyses on Google's infrastructure. One of the ways to interact with this platform is through the Code Editor, which is an integrated development environment that allows users to write and run scripts using the earth engine JavaScript API from their browser. *This means users can access earth data and perform geospatial analyses with a few lines of codes using only a web browser and Earth Engine access.*

Who can access Earth Engine?

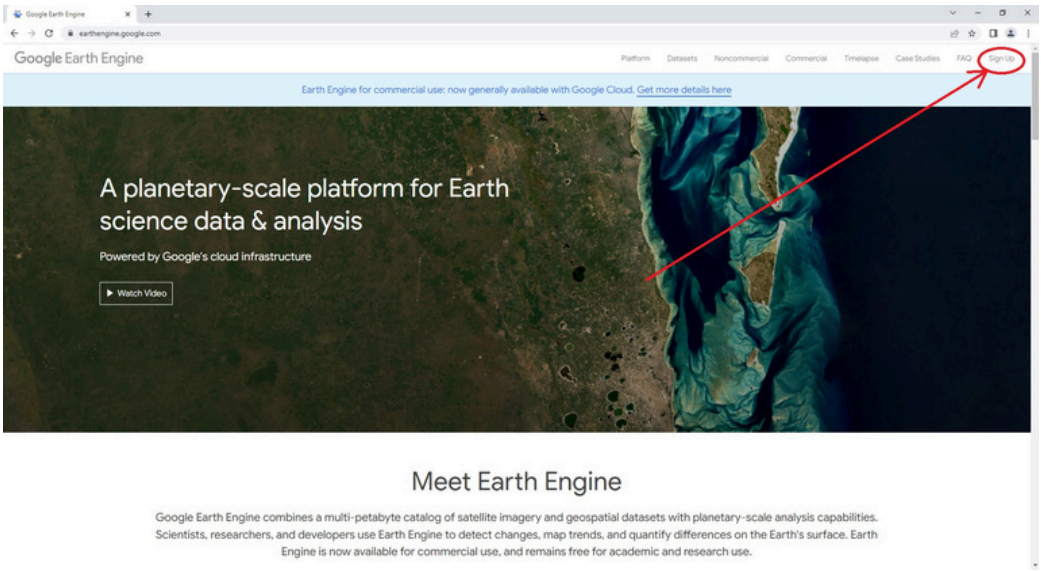
Earth Engine access is granted for *free* for development, research, educational and humanitarian use. However, to enable Earth Engine access, you will need to *sign up and register your account* by providing some relevant information. The approval process might take a few days and sometimes a week! We recommend registering ahead so your account will be all set at the time of the workshop.

How to sign up/set Earth Engine access?

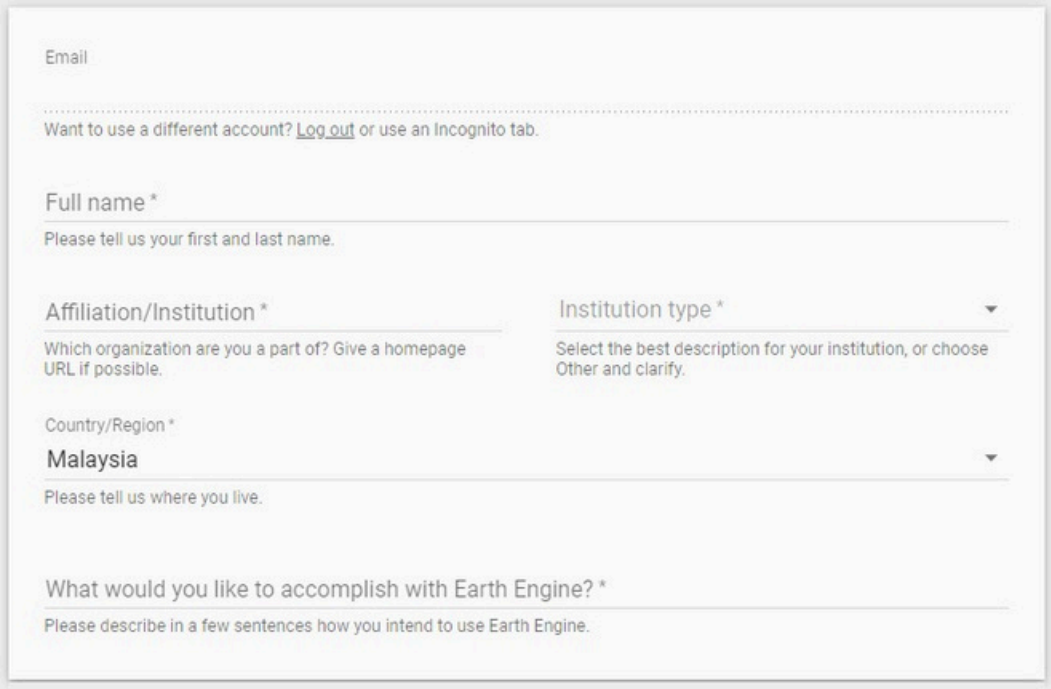
To set your Earth Engine access, you will only need an internet access and a Google Chrome browser(If you don't have Chrome already,clickhereto download it).

- Open Google Chrome browser and go toearthengine.google.com

- Click on the Sign up button in the upper right corner.



- Once directed to the sign up page, you will be asked to provide some basic information about your institution and how you intend to use Earth Engine. It is recommended that you fill up all the fields with clear information.

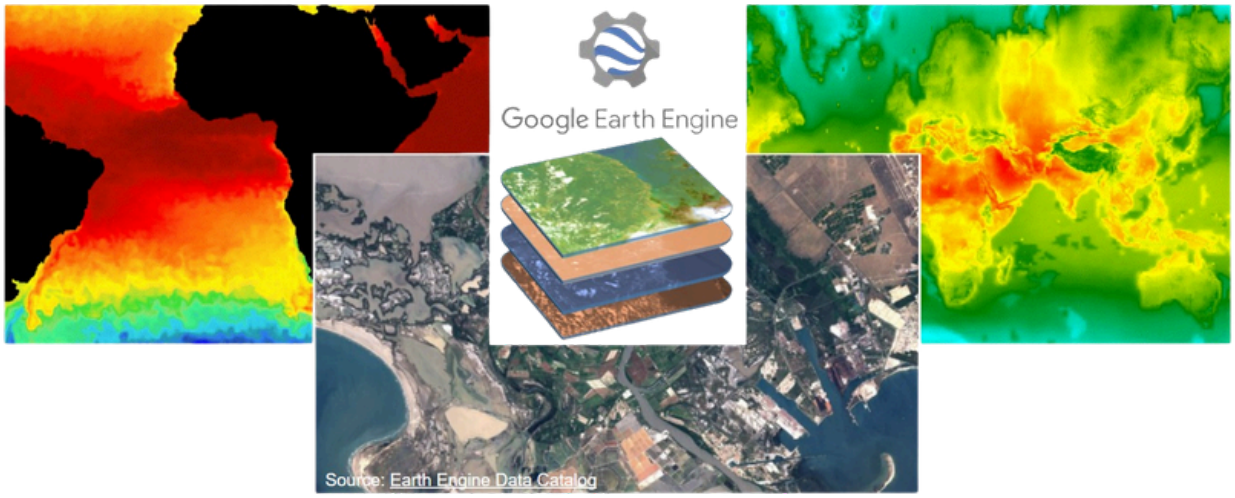


- A verification email will be sent to your email address once your account is enabled for Earth Engine access, typically, between two days to a week from registration.

Can we do more in the meantime?

In the meantime, you can start exploring the dataset available in the *Earth Engine Data Catalog* and check if your favorite dataset is there. You can use the tag search for browsing the data by category as well.

Additionally, you may also check the web interface Earth Engine Data Explorer where some of the data from the catalog are available for an interactive view and basic analysis outside of the code editor environment.



Introduction to Earth Engine

Introduction to GEE
Lab#1

1. Introduction to Google Earth Engine

1.1 Rational & Advantages

- Petabytesofremotelysenseddataarenowpubliclyavailableincluding+40years of archived satellite images, elevation models, and a wide range of climatic and environmental data.
- Dataset are updated continuously and new data is being added everyday
- The analysis ready products are increasingly available
- GEE helps with addressing the need for an advanced geospatial processing platform to process big data and provide easy access to the large archive of remotely-sensed images
- GEE can support the work related to large scale earth data and global issues such as drought, deforestation, natural disasters, disease, food security, water resources, climate monitoring, environmental monitoring, and many others workflow includes automation of data processing, near real time monitoring, machine learning applications

1.2 Scope

- This workshop provides basic hands-on experience using Earth Engine and links users with the various available resources to continue learning. For information on the GEE platform background, you may start with this article by Noel Gorelick et al. (2017)



- The code editor environment usingJavaScriptAPIwillbe the main focus of this workshop, users will have hands-on experience to work on some fundamental data preparation and analysis workflow that can be transferred to their projects in many fields related to environmental earth sciences.



- The workshop provide steps that are needed to start your own remote sensing imagery project according to your research interest using Earth Engine

- One other way tointeractwithearthengineoutsideofthe code editor environment is to use the Google colaboratory (Google Colab) using Python API. The colab and authentication to earth engine can be set fromhere.



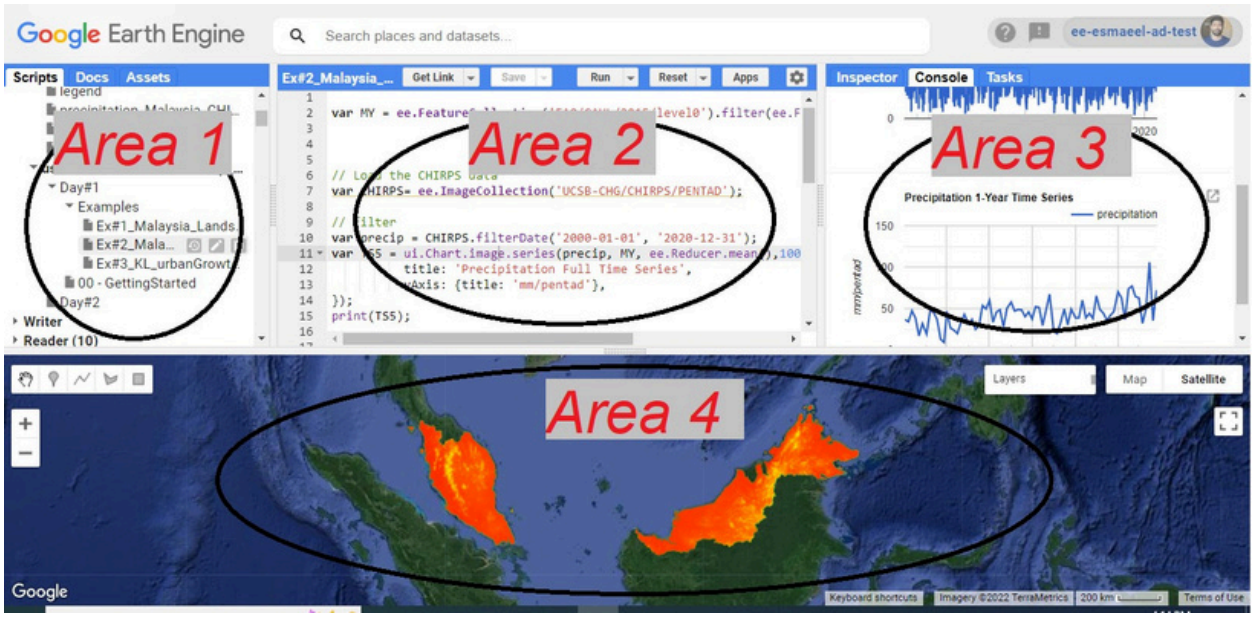
1.3 Signing up & setting an account

- Recap of Lab#0: Checking Earth Engine access for all

1.4 Getting started with GEE: GEE interface & code editor

- Open the code editor environment https://code.earthengine.google.com/
    - Four main adjustable areas:

      Area 1: Scripts, documentation and data
      Area 2: Code editor; write and run the scripts
      Area 3: Results; inspect the map, print (chart & results), export the data
      Area 4: Results; Visualize on map, drawing points, line, polygons



    - For reference info on the interface and code editor: refer to the Code editor doc

    - Explore Area 1: The script manager
      -Notice where the scripts are saved, look at the default folders; reader and examples
      -Check the documentation tab where you can find the functions
      -Check the assets tab where imported files and cloud folder are located
      -Check the scripts tab again and open an example form the default example folder and notice how the script appears in the Area 2

    - Explore Area 2: The code editor;

-Run different scripts example
-Save one script and check it in the Area 1 in the scripts' tab

- Import the workshop scripts repository by clicking on the following linkIPI_ workshop repository
  -Check the scripts in the new scripts appeared in the reader folder

- Open and run scripts examples from the IPI workshop imported repository



Malaysia Landsat 8 composite/Mosaic



Malaysia precipitation, time Series and temperature



Time lapse: Urban Growth KualaLUmpur

- Explore Area 3:
  -Check the console tab
  -Run Ex_2 and check the chart view in the consol and the interactive web view
  - Check the interactive inspector using Ex_1 & Ex_2

- Explore Area 4: maps, layer manager, visualization options (Ex_1 & Ex_2)
  -Check the map; pan, zoom in and out
  -Check the layers manager and visualization
  -Check the interactive drawing tools and the geometry manager tab
  -Check the inspector tab in Area 3 gain with the Map

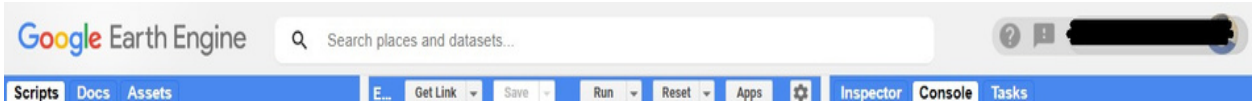
- Explore Time series: External links with (Ex_3)

- Explore default scripts and examples

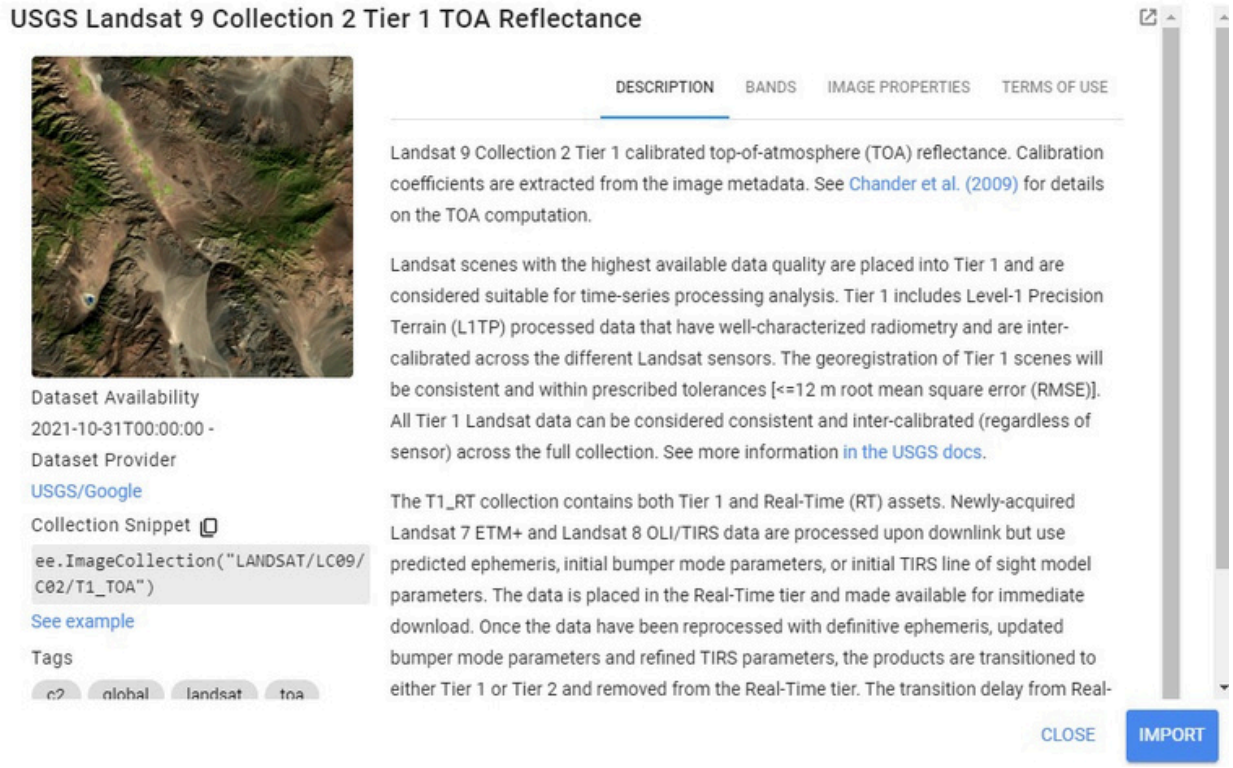
1.5 GEE data catalog & dataset access

- Openthe GEE data catalogExplore the data available

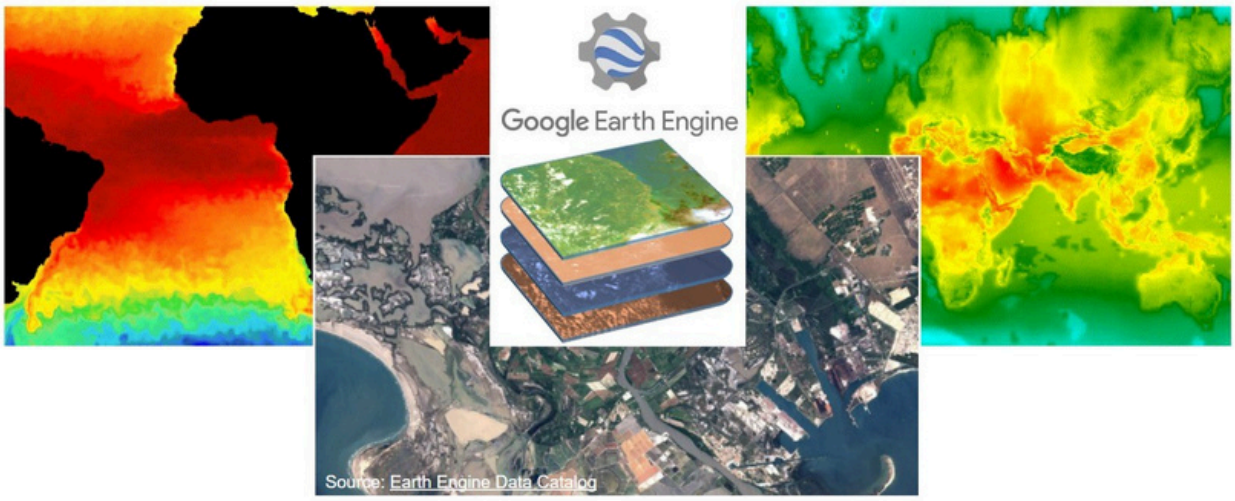
- Use the search bar in the GEE code editor environment to access data from the catalog: Ex. search for digital elevation model SRTM



- Explore data organization; description, citation, script example
  Ex. Search and explore landsat 9



- Identify data path (identifier), type, and other ways to import



Introduction to Earth Engine

JavaScripts and Spatial data in GEE
Lab#2

2. JavaScript and Spatial data in Google Earth Engine

2.1 JavaScripts basics & Syntax

Open the code editor environment https://code.earthengine.google.com/

- Comments: `//` and `/* .... */`
    - Write comments

```
// This is a comment
```

    - Write comment section

```
 /* This is a comment section
and It will not be executed
 */
```

- Introducing variables: `var variable_name = ...`
    - String

```
vartextname ="the text content";
```

    - Number as variable and print to console

```
varx =10;
print(x);
```

- Add title to the printed objects

```
print("The text content", textname);
print("The variable x", x);
```

- Creating lists

```
// list of names or list of number
varSomeSatellites = ['Landsat','Sentinel','MODIS'];
varSomeNumbers = [8,2,1];
```

- Printing lists / select and print

```
print("Printing entire list", SomeSatellites);
print("printing the element number 0 of the list",
SomeSatellites[0]);
```

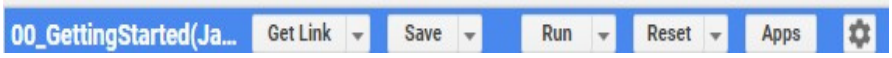
- Creating dictionaries / select and print

```
//Dictionaries    i.e.    names    and    values
varNamesAndValues  =  {'Landsat':'Landsat8  30m
resolution','Sentinel':'10 m','MODIS':250};

// print the dictionary
print(" the whole dictionary", NamesAndValues);

// Print Only one object
print("object titled Landsat", NamesAndValues['Landsat']);
//or
print("object titled Landsat", NamesAndValues.Landsat);
```

- Save the script ; and get link to share the script

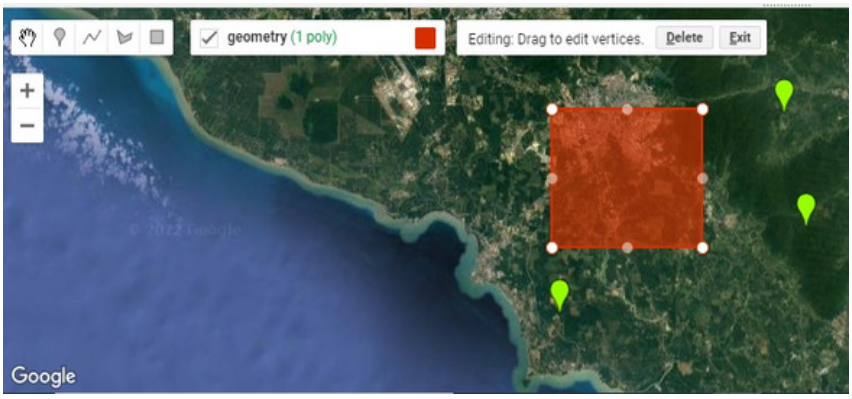


- All used commands are available in the Workshop-IPI22 repository part 1: *00_GettingStarted(Java_variables&Syntax)*

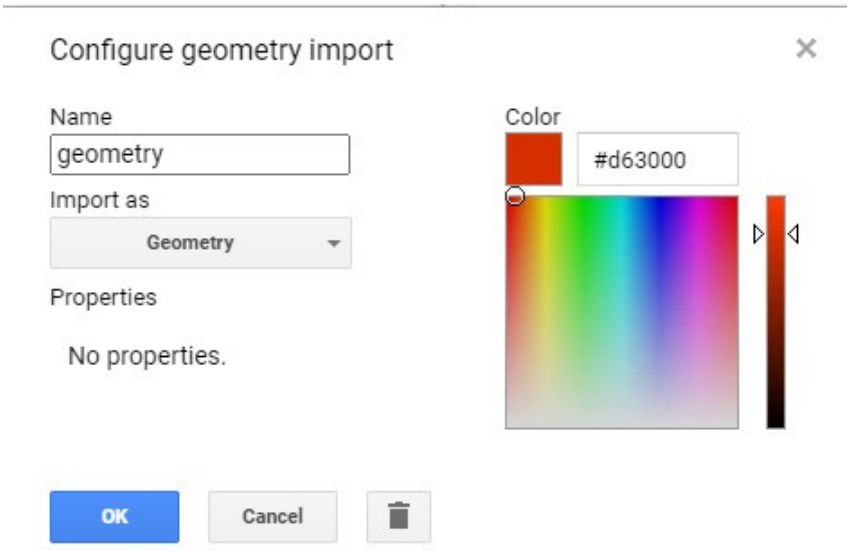
2.2 Spatial data and terminology in GEE

2.2.1 Vector data: geometry, Feature, FeatureCollection

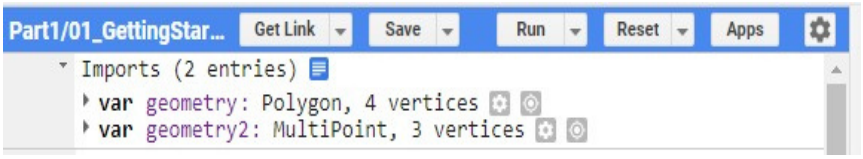
- Adding geometry objects interactively to the map
    - Adding points, lines, and polygons



    - Changing colors, location, delete



    - Noticing the code editor import section and the geometry manager; change names, and delete from there



- Map with the code editor; Adding geometry objects using coordinates
    - ○Inspect coordinates using the interactive inspector…
    - ○Center the map accordingly `Map.setCenter(lon, lat,zoom)`
      Use Space + Entertoautofill thepossible arguments

```
//Center the map using coordinates and specify a zoom level
Map.setCenter(101.771,2.920, 15);
```

    - Creating point `ee.Geometry.Point()`

```
// Creating point geometry object
varpoint = ee.Geometry.Point(101.771,2.920);

//print the point!!
print("point", point);
```

    - Add the point to the map

```
//Add the point to the map
Map.addLayer(point);
```

    - Creating multipoints `ee.Geometry.MultiPoint()`Andadd to map with a layer name

```
//Creating multi points
varpoints = ee.Geometry.MultiPoint(101.756,2.958,
        101.756,  2.889,
        101.826,  2.889,
        101.826, 2.958);

// Add this layer to the map and name it
Map.addLayer(points, {},"Multi points");
```

- Line `ee.Geometry.LineString()` And add to map with a layer name and chosen color

```
//Lines...
varline = ee.Geometry.LineString(101.756,2.99,
          101.73, 2.88);
// Add the line to the map; name it and change its color to red
Map.addLayer(line, {color:"red"},"line");
```

- Polygon `ee.Geometry.Polygon()`

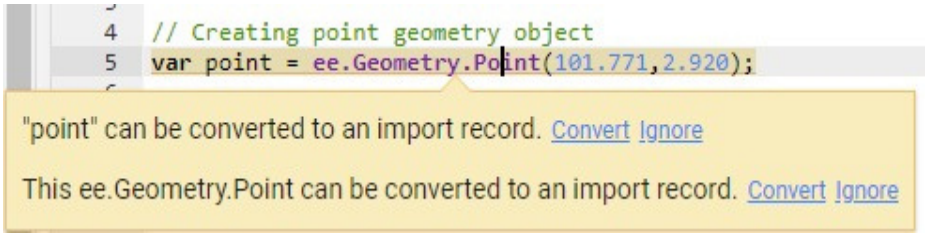
```
//polygons...
varpolygon = ee.Geometry.Polygon(101.756,2.95,
        101.75,     2.88,
        101.82,    2.889,
        101.826, 2.958);

// Add the polygon to the map; name it and make it blue
Map.addLayer(polygon, {color:"blue"},"polygon");
```

- Center using object this time `Map.centerObject()`

```
//Centering the map on object
Map.centerObject(polygon);
```

- Import a geometry object to the import recorde by hovering and clicking import



- Note: All map related arguments started with `Map.`

- Note: when you see commands with " ee. " this is a server object… bear this in mind for later

- Features are Geometry with attributes (properties): the basic vector object in GEE
    - Creating feature from geometry and populate properties `ee.Feature()`

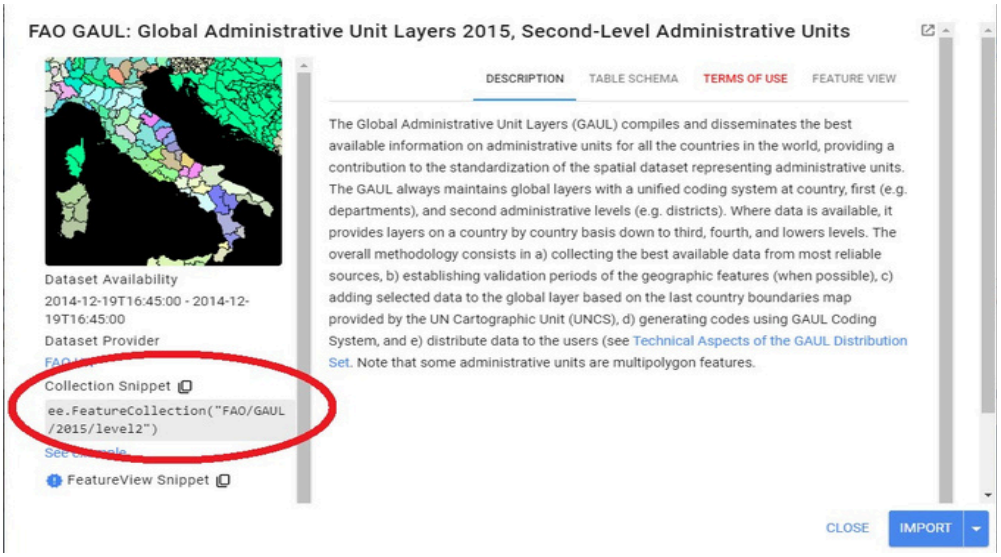
```
// Let's make the previous polygon a feature and add properties
varfeature = ee.Feature(polygon, {'polygon id':25,'Name':"UKM surrounding"});

// Add the feature to the map; name it and color it orange,
Map.addLayer(feature, {color:"orange"},"feature");
```

    - Comparing feature and polygon on the map and in console

```
//print polygon and feature and compare in the console
print("the polygon", polygon);
print("the feature", feature);
```

- Feature collection is a container to host a collection of features
    - Search, check and add FAO GAUL feature collection from the catalog




```
//Add for FAO GAUL check the info, bands, and copy the path
varfc = ee.FeatureCollection("FAO/GAUL/2015/level0");

//Check the entire world
Map.addLayer(fc);//zoom out
```

- Use `.filter(ee.Filter.eq(name, value))` to filter the feature collection by property value to Malaysia boundary

```
//filter feature collection to malaysia
varMalaysia = ee.FeatureCollection("FAO/GAUL/2015/level0")
                    .filter(ee.Filter.eq('ADM0_NAME','Malaysia'));

Map.addLayer(Malaysia, {},"Malaysia Boundary");

//Centering the map
Map.centerObject(Malaysia);
```

- Convert Malaysia boundary to geometry and check the properties for both

```
//Make it Geometry (some functions take only geomtery as an input)
varMY = Malaysia.geometry();

//Compare in consol
print("Malaysia feature collection", Malaysia)
print("Malaysia geometry", MY)
```

- All used commands are available in the Workshop-IPI22 repository part 1: *01_GettingStarted(vector data)*

2.2.1 Raster data: Image, ImageCollection

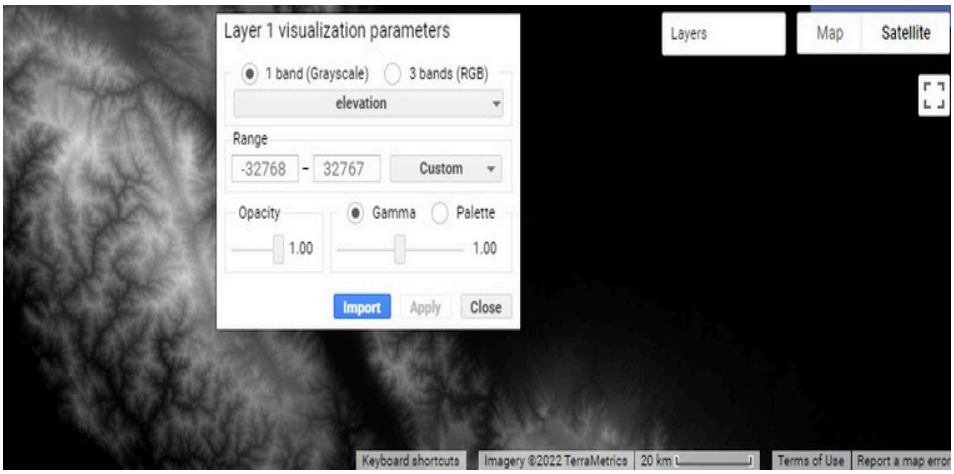
- Images are a stack of georeferenced bands with metadata
    - Import single band image: search for SRTM in the catalog and import

```
//Add image SETM
varDEM = ee.Image("USGS/SRTMGL1_003");

//Center Map
Map.setCenter(101.8, 2.9, 5);

//Add to the map
Map.addLayer(DEM)
```

    - Inspect the image and Visualize it using the interactive tool/layer manager



    - Visualize image; set min and max in the Map.addLayer()

```
//Add image to the map
Map.addLayer(DEM, {'min':0,'max':4000},"SRTM_DEM");
```

    - Add custom visualization with palette and add layer again

```
//Custom visualization and add to the map
varviz = {'min':0,'max':4000,'palette':"white,brown,green,red"};
Map.addLayer(DEM, viz,"SRTM_DEM_palette");
```

- Clip Image to a region of interest

```
//Draw geometry to clip to region of interest
varroi = ee.Geometry.Polygon(102,3,
          102, 2.5,
          102.5, 2.5,
        102.5, 3);

//Clip image
varC_DEM = DEM.clip(roi);

//Add clipped image to the map
Map.addLayer(C_DEM, viz,"Clipped SRTM_DEM");
```

- Adjust the min and max in the customized visualization and add the clipped image again

```
varviz2 = {'min':0,'max':1000,
'palette':"white,brown,green,red"};
Map.addLayer(C_DEM, viz2,"Clipped SRTM_DEM2");
```

- Comments the layers you don't want or add the argument false to Map.addLayer

```
//Map.addLayer(DEM, {'min': 0, 'max': 4000}, "SRTM_DEM");
varviz = {'min':0,'max':4000,'palette':"white,brown,green,red"};
Map.addLayer(DEMviz, viz,"SRTM_DEM_palette",false);
```

- Rename the single band image using.rename()

```
// inspect and change band name
varDEM = C_DEM.rename("DEM");

Map.addLayer(DEM, viz,"new band name");

Map.centerObject(DEM, 10);
```

- All used commands are available in the Workshop-IPI22 repository part 1: *02_GettingStarted(raster data)*

- ImageCollection is a container of images; a time series. Filtering imageCollection
    - Add administration boundary and filter to region of interest

```
//Add administration boundary and filter to region of interest
varSelangor =
ee.FeatureCollection('FAO/GAUL/2015/level1').filter(ee.Filter.eq('ADM
1_NAME','Selangor'));

// center the map
Map.setCenter(101.8, 2.9, 10);
```

    - Search the Landsat 8 T1_L2 collection.. Add and Print the collection

```
//Get Landsat collection
varL8_collection = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2');

//add to the map
Map.addLayer(L8_collection,{},"Entire collection");

//print
print(L8_collection, "L8_collection");
```

    - Filter 1 to the region of interest, print and add to the map

```
//Filter to the region of interest
varFiltered1 = L8_collection.filterBounds(Selangor);

// print and add to the map
print(Filtered1, "Filtered1");

Map.addLayer(Filtered1,{},"Filtered to region");
```

    - Filter 2 to a date range of interest, print and add to the map

```
//Filter for date range
varFiltered2 = Filtered1.filterDate('2020-06-01','2020-12-31');

// print and add to the map
print(Filtered2, "Filtered2");
Map.addLayer(Filtered2, {},"Filtered to date and region");
```

- Compare the filtered collections and original collection in console

- Sort by cloud cover and get the first image, add it to map with RGB

```
// sort by cloud cover and get the first image
varsorted= Filtered2.sort("CLOUD_COVER");
varfirst = sorted.first();

// print and add to the map
print(first,"first image");
Map.addLayer(first, {bands: ['SR_B4','SR_B3','SR_B2'],min:0, max:
30000}, 'first');
```

- Make a median composite from sorted collection

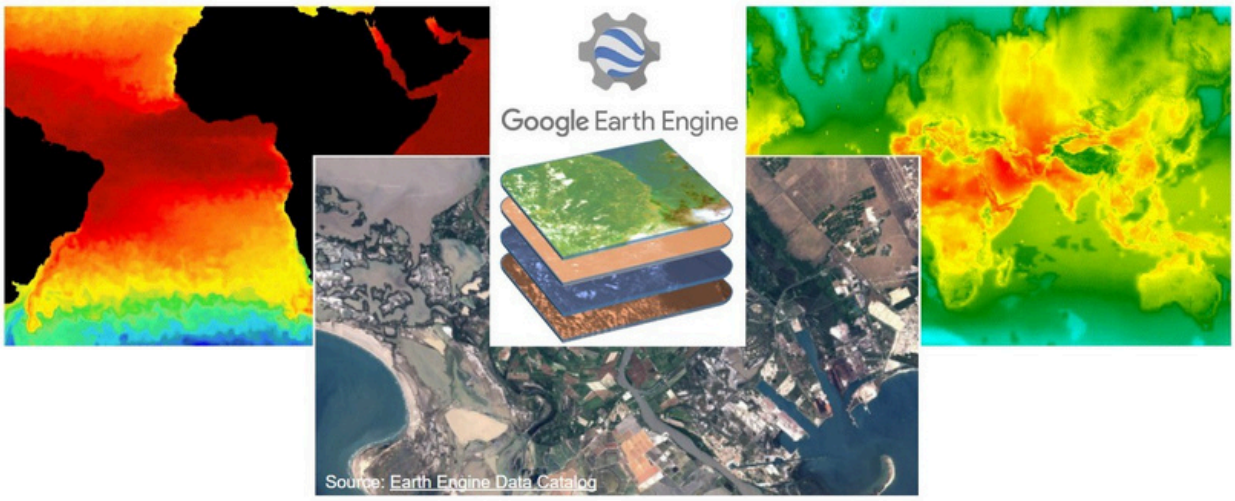
```
//Composite median
varCompositeMedian = sorted.median();

// print and add to the map
print(CompositeMedian, "CompositeMedian");
Map.addLayer(CompositeMedian, {bands: ['SR_B4','SR_B3','SR_B2'],
min:0, max:30000},'CompositeMedian');
```

- Clip and add again

```
// Clip and add again
varCompositeMedian = CompositeMedian.clip(Selangor);
Map.addLayer(CompositeMedian, {bands: ['SR_B4','SR_B3','SR_B2'],
min:0, max:30000},'CompositeMedian');
```

- All used commands are available in the Workshop-IPI22 repository part 1: *03_ImageCollection*



Introduction to Earth Engine

Operations on Images and ImageCollections
Lab#3

3. Operations on Images and ImageCollections

3.1 Composite recap, bands manipulation, NDVI

- Filter the collection in on command, make a median composite

```
//Add administration boundary and filter to region of interest
varSelangor =
ee.FeatureCollection('FAO/GAUL/2015/level1').filter(ee.Filter.eq('ADM
1_NAME','Selangor'));
Map.setCenter(101.8, 2.9, 10);

//Get Landsat8 collection and filter it as in previous lab
varFiltered = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2')
                  .filterBounds(Selangor)
                  .filterDate('2020-06-01', '2020-12-31')
                  .sort("CLOUD_COVER");

//Composite median
varCompositeMedian = Filtered.median();
```

- Inspect the image, and make RGB image from it by selecting RGB bands

```
// making RGB image
// specify the RGB bands names and put them into list
varbands = ['SR_B4','SR_B3','SR_B2'];
// Select RGB image using the list of name
varRGBImage = CompositeMedian.select(bands);

// Add to layer and inspect
print(RGBImage,"RGBImage : three bands image");
Map.addLayer(RGBImage, {min:0, max:30000},"RGBImage");
```

- Inspect the RGB image, and change bands name using.select()

```
//change band name using select
varnewnames = ["Red","Green","Blue"];
varRGB = CompositeMedian.select(bands, newnames);

// add and inspect
Map.addLayer(RGB, {min:0, max:30000},"RGB with colors'names");

// add and clip to selangor within Map.addLayer()
Map.addLayer(RGB.clip(Selangor), {min:0, max:30000},"RGB Selangor");
```

- Calculate NDVI from the median composite (CompositeMedian)

```
// NDVI = (NIR - R) / (NIR + R)
var NDVIimage=
CompositeMedian.normalizedDifference(['SR_B5','SR_B4']);

Map.addLayer(NDVIimage, {min:-1, max:1, palette: ['white','green']},
"NDVIimage");

// Inspect the image and rename the band to 'NDVI'
varNDVIimage = NDVIimage.rename('NDVI');

//Add to the map and improve the palette by adding blue
Map.addLayer(NDVIimage, {min:-1, max:1, palette: ['blue','white',
'green']}, "NDVIimage");
```

- Add the NDVI image to the median composite (CompositeMedian)

```
//Add the NDVI image to the median composite (CompositeMedian)
varCompWIthNDVI = CompositeMedian.addBands(NDVIimage);

//Add to the map (select NDVI to visualize) and inspect
Map.addLayer(CompWIthNDVI, {bands: ["NDVI"], min:-1,max:1, palette:
['blue','white','green']},"Composite WIth NDVI");
```

3.2 Writing and applying/mapping functions

- Functions are used to automate and organize operation; it take an input and make a few operation/calculation and return an output

General syntax:

```
varfunctionName= function(input){

  do something with the input (operation);

  return output;
};
```

To apply the function:
var product = functionName(new input)

- Function to calculate and add NDVI to Landsat_SR image

```
// function to calculate and add NDVI to an image
var addNDVI= function(image){
  var ndvi=image.normalizedDifference(['SR_B5','SR_B4']).rename('NDVI');
  return image.addBands(ndvi);
};

// Apply the function on the previous composite
varCompWIthNDVI2= addNDVI(CompositeMedian)

//Add to the map (select NDVI to visualize) and inspect
Map.addLayer(CompWIthNDVI2, {bands: ["NDVI"], min:-1,max:1, palette:
['blue','white','green']},"Composite WIth NDVI fromfunction");
```

- Mapping a function means to apply it on every single feature in a collection using `.map()`
    - Calculate and add NDVI for every single image in the filtered collection (Filtered) using the previous function

```
//Calculating and adding NDVI for every single image in the filtered collection (Filtered)
varCollectionwithNDVI = Filtered.map(addNDVI);

//add and inspect
Map.addLayer(CollectionwithNDVI, {bands: ["NDVI"],min:-1, max:1, palette: ['blue','white','green']},"Collection withNDVI");
```

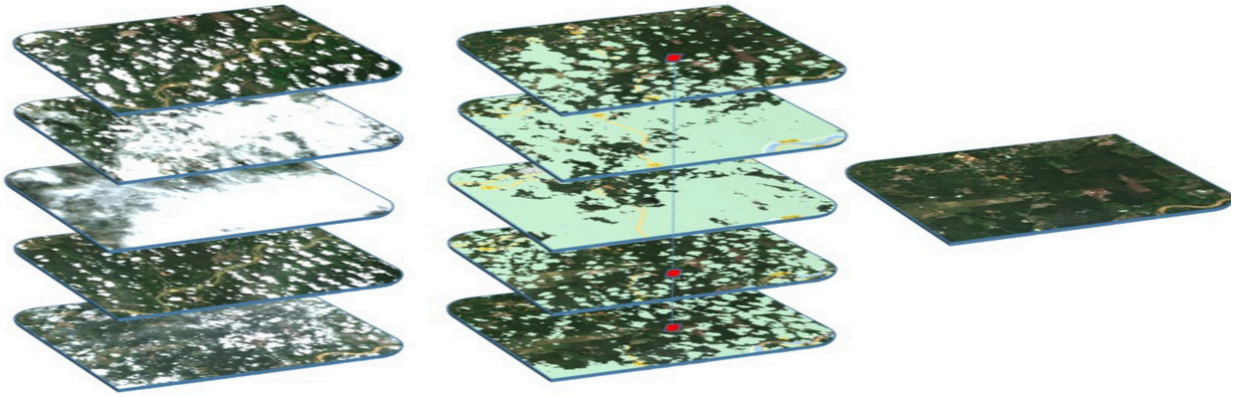
    - Making a median composite NDVI

```
// make a median NDVI
varmedianNDVI = CollectionwithNDVI.median().select("NDVI")
Map.addLayer(medianNDVI, {bands: ["NDVI"], min:-1,max:1, palette: ['blue','white','green']},"median NDVI");
```

- .map()is similar to a*loop*, it apply a function onevery single feature of a collection

- All used commands are available in the Workshop-IPI22 repository part 1: *04_OperationOnImagesAndCollections*

3.3 Cloud free composite (Landsat 8 example)



- To get a cloud free composite, we mask the cloud out of a collection and composite the collection

- To identify the unwanted pixels when working with Landsat 8 science products (Collection 2, Level 2); the surface reflectance data is analysis ready data with additional bands to identify clouds, cloud shadows, and snow/ice derived by applying CFMask algorithm by NASA. Additional bands (e.g saturation bands “QA_RADSAT”) are also available. Check the Landsat 8 science product guid herefor more information on the CFMask and the qualitybands .

- The CFMask function use the QA_PIXEL band and saturation band QA_RADSAT and apply scaling factors specific to the bands to score the image

    - Prepare and filter a collection to date/region as in the previous work

```
//Add administration boundary and filter to region of interest
varSelangor =
ee.FeatureCollection('FAO/GAUL/2015/level1').filter(ee.Filter.eq('ADM
1_NAME','Selangor'));

Map.setCenter(101.8, 2.9, 10);

//Get Landsat8 collection and filter it as in previous lab
varFiltered = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2')
                  .filterBounds(Selangor)
                  .filterDate('2020-06-01', '2020-12-31')
                  .sort("CLOUD_COVER");
```

- Copy and paste cloud masking function under the previous script

```
//###########################################################
//            ## Copy paste the following function ##
//###########################################################

// This example demonstrates the use of the Landsat 8 Collection 2,
Level 2 QA_PIXEL band and QA_RADSAT band (CFMask) to mask unwanted
pixels

functionmaskL8sr(image) {
  varqaMask = image.select('QA_PIXEL').bitwiseAnd(parseInt('11111',
2)).eq(0);
  varsaturationMask = image.select('QA_RADSAT').eq(0);

  // Apply the scaling factors to the appropriatebands.
  varopticalBands =
image.select('SR_B.').multiply(0.0000275).add(-0.2);
  varthermalBands =
image.select('ST_B.*').multiply(0.00341802).add(149.0);

  // Replace the original bands with the scaled onesand apply the
masks.
  return image.addBands(opticalBands, null, true)
      .addBands(thermalBands, null, true)
      .updateMask(qaMask)
      .updateMask(saturationMask);
};
//###########################################################
//            ## End of the function ##
//###########################################################
```

- Map the function over the filtered collection and make a median composite of the masked images

```
// Map the function over the collection
varCloudFreeCollection = Filtered.map(maskL8sr);
// Composite median and Clip to the output
varCompositeCloudFree = CloudFreeCollection.median().clip(Selangor);
```

```
// Display the results
Map.addLayer(CompositeCloudFree, {bands: ['SR_B4','SR_B3','SR_B2'],
min:0, max:0.3},'CompositeCloudFree');
```

3.4 Exporting image

- Minimize size and exported pixels; Select relevant bands only and clip

```
// select relevant bands only
varbands = ['SR_B2','SR_B3','SR_B4','SR_B5','SR_B6','SR_B7'];
varFinalImage = CompositeCloudFree.select(bands);

// make a small region of interest and clip the image to it
varroi = ee.Geometry.Polygon(101.756,2.95,
          101.75, 2.88,
          101.82, 2.889,
         101.826, 2.958);

varFinalImage = CompositeCloudFree.clip(roi);

// Display the final results
Map.addLayer(FinalImage, {bands: ['SR_B4','SR_B3','SR_B2'], min:0,
max:0.3},' Final CompositeCloudFree');
Map.setCenter(101.75, 2.88, 12);
```

- Export to drive

```
// Export
Export.image.toDrive({
image: FinalImage,
description: 'roi_cloudFree',
scale: 30
});
```

- All used commands are available in the Workshop-IPI22 repository part 1: *05_CloudFreeComposite_Landsat8*

3.5 Practical tip and (Sentinel 2 cloud free composite example)

- For data preparation tasks (Such as preparing cloud free composite and adding indices, selecting bands….), it's recommended to assemble your own functions to call and prepare the data at the beginning of your code

- The following is a pre-written function to prepare Sentinel-2 ***cloud free median composite*** and add indices in one line. It uses the cloud probability method, which is one of the best methods to mask clouds and shadows from Sentinel-2 images.

- The function default arguments are:

```
GetS2CloudFree(StartDate, EndDate, cloudThreshold, region);
```

- Applying the function and getting the cloud free composite

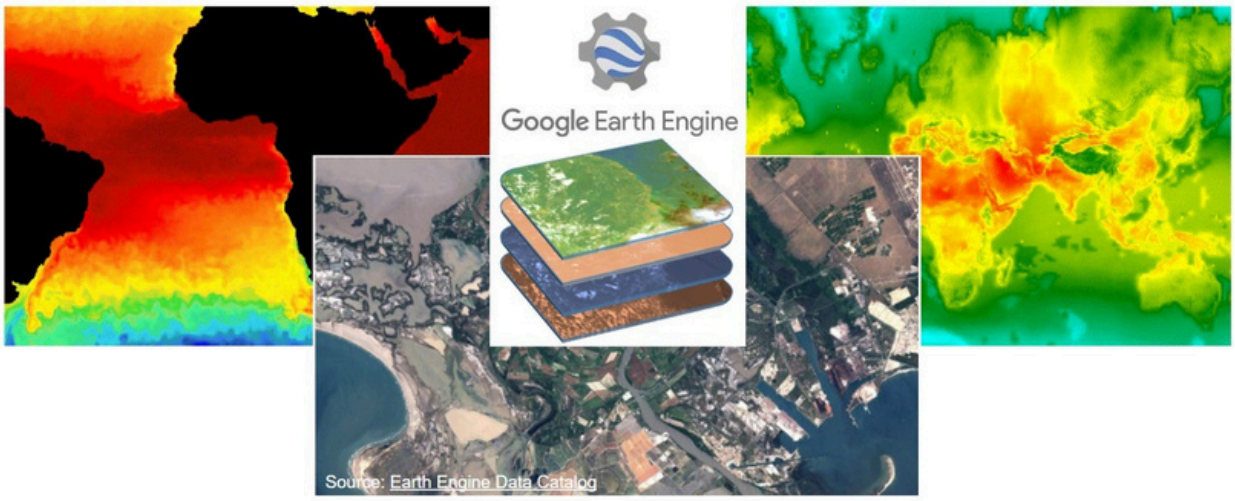
```
varroi = ee.Geometry.Polygon(101.756,2.95,
        101.75,    2.88,
        101.82,   2.889,
        101.826, 2.958);

// call the function and set parameters StartDate, EndDate,
cloudThreshold, region
varCloudFreeS2 = GetS2CloudFree('2020-05-01','2020-09-01',40,
roi);

// set vizualization
varTColorS2Vis = {'bands': ['B4','B3','B2'],'min':0,'max':
3000};

Map.addLayer(CloudFreeS2, TColorS2Vis ,'Cloud FreeS2');
Map.centerObject(roi);
```

- All used commands are available in the Workshop-IPI22 repository part 1: *06_CloudFreeComposite_Sentinel2*



Introduction to Earth Engine

Band math and Images Reductions
Lab#4

4. Band math and Images Reductions

4.1 Band math and masking with indices

- Prepare Landsat 8 Cloud free median composite over a region of interest as in the previous lab

```
//############################################################
//            ## functions ##
//############################################################
// function to mask cloud Landsat 8
functionmaskL8sr(image) {
  varqaMask = image.select('QA_PIXEL').bitwiseAnd(parseInt('11111',
2)).eq(0);
  varsaturationMask = image.select('QA_RADSAT').eq(0);

  // Apply the scaling factors to the appropriatebands.
  varopticalBands =
image.select('SR_B.').multiply(0.0000275).add(-0.2);
  varthermalBands =
image.select('ST_B.*').multiply(0.00341802).add(149.0);
```

```
  // Replace the original bands with the scaled ones and apply the
masks.
  return image.addBands(opticalBands, null, true)
      .addBands(thermalBands, null, true)
      .updateMask(qaMask)
      .updateMask(saturationMask);
}
//###########################################################
//            ## End of the function ##
//###########################################################
//Add administration boundary to region of interest
varSelangor =
ee.FeatureCollection('FAO/GAUL/2015/level1').filter(ee.Filter.eq('ADM
1_NAME','Selangor'));

Map.setCenter(101.8, 2.9, 10);

//Get Landsat8 collection and filter it as in previous lab
varFiltered = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2')
                  .filterBounds(Selangor)
                  .filterDate('2020-01-01', '2020-12-31')
                  .sort("CLOUD_COVER");

//get cloud free median composite
// Map the function over the collection, Composite median and Clip to
the output in one line
varCFComposite = Filtered.map(maskL8sr).median().clip(Selangor);

// Display the results
Map.addLayer(CFComposite, {bands: ['SR_B4','SR_B3','SR_B2'], min:
0, max:0.3},'CloudFree Composite');
```

- Get an NDVI image from the cloud free composite

```
// Get NDVI image from the cloud free composite
varNDVIimage =
CFComposite.normalizedDifference(['SR_B5','SR_B4']).rename('NDVI');
Map.addLayer(NDVIimage, {min:-1, max:1, palette: ['blue','white',
'green']}, "NDVIimage");
```

- Use threshold 0.7 to identify healthy vegetation pixel

```
//filter NDVI image to specific threshold , Mask
//return a binary image 0 and 1
varVegMask = NDVIimage.gt(0.7);

Map.addLayer(VegMask,{ min:0, max:1, palette: ['white','green']} ,
'Vegetation mask');
```

- Use the previous mask to mask (clip) NDWI to healthy veg only

```
//use it to mask the NDVI image
varVeghealthy = NDVIimage.updateMask(VegMask);

Map.addLayer(Veghealthy, { min:0.7, max:1, palette:['white',
'green']},'Only Vegetation NDVI');
```

- Get an NDWI image from the cloud free composite

```
// Get Normalized Difference Water Index: NDWI = (G-NIR)/(G+NIR)
varNDWIimage =
CFComposite.normalizedDifference(['SR_B3','SR_B5']).rename('NDWI');
Map.addLayer(NDWIimage, { min:-1, max:1, palette:['brown',
'white','blue', 'blue']}, 'NDWIimage');
```

- Use threshold 0 to identify water pixel

```
//filter NDWI image to specific threshold , Mask
//return a binary image 0 or 1
varwaterMask = NDWIimage.gt(0);

Map.addLayer(waterMask,{ min:0, max:1, palette:['white','blue']}
,'water mask');
```

- Use the previous mask to mask (clip) NDWI to water only

```
//use it to mask the NDWI image
varwater = NDWIimage.updateMask(waterMask);

Map.addLayer(water, { palette: ['blue']},'Only WaterNDWI');
```

- Combine indecs masks to classify the images based on the thresholds

```
// indices Masking
varWaterClass = waterMask.rename("Class");

//do some math (image with numbers)
varVegClass = VegMask.multiply(2);

// do math (image with image, pixel to pixel math)
varindicesClasses = WaterClass.add(VegClass);

Map.addLayer(indicesClasses, { min:0, max:2, palette:['red',
'blue','green']}, 'indicesClasses');
```

- All used commands are available in the Workshop-IPI22 repository part 1: *07_BandMathAndMaskingIndices*

4.2 Image statistic & reductions

From previous workflow, we have both the `Filtered` collection and the `CFComposite`

- Introduce small roi and clip CFComposite accordingly and make NDVI imageout over the roi from the composite (using the function as in the previous workflow)

```
//make a small region as roi
varroi = ee.Geometry.Polygon(101.756,2.95,
        101.75,    2.88,
        101.82,   2.889,
        101.826, 2.958);
varCFComposite = CFComposite.clip(roi);
//Add NDVI to the cloud free composite and select the NDVI image
varNDVIimage = addNDVI(CFComposite).select("NDVI");
```

- Clip/mask NDVI to the healthy vegetation only (as in the previous workflow)

```
varmaskveg = NDVIimage.gt(0.7);
Map.addLayer(maskveg, { min:0.7, max:1, palette:['white',
'green']}, 'mask');

// get healthy veg in this region
varhealthyVeg = NDVIimage.updateMask(maskveg);
```

- Apply spatial reduction by reducing NDVI image pixel value to the average value over the roi (calculate the NDVI statistic (mean) in the roi)

```
// reduction of pixel value
varaverage = healthyVeg.reduceRegion(ee.Reducer.mean(),roi,30);

print("average NDVI value",average);
```

- Notice the output of previous reduction is a dictionary

- Apply spatial reduction to calculate the area based on the pixels

```
// region area
varpixelarea = ee.Image.pixelArea().divide(1000000);

vararea = pixelarea.reduceRegion({
                geometry: roi,
                reducer: ee.Reducer.sum(),
                scale: 30});
 print("area in sq km", area);
```

- Notice the output of previous reduction is a dictionary

- Apply Neighborhood reduction to calculate the mean neighborhood statistic of an NDVI image

```
//reduce neighborhood
varneighborhood = healthyVeg.reduceNeighborhood(
  ee.Reducer.mean(),
  ee.Kernel.square({radius: 2}));
```

```
Map.addLayer(neighborhood, { min:-1, max:1, palette:['white',
'green']}, 'neighborhood', false);
```

- Notice the output of previous reduction is an image
    - To chart NDVI over time over the healthy area of the roi, we need first to clip/mask all NDVI image in the collection to the extent of the healthy vegetation; we use mask based on threshold and map it over collection

```
// chart NDVI

// add NDVI to collection and mask the collection to healthy veg
var healthyMask= function(image){

  return image.updateMask(maskveg);
};

varhealthycol =
Filtered.map(addNDVI).select("NDVI").map(healthyMask);


Map.addLayer(healthycol, { min:-1, max:1, palette:['white',
'green']},'NDVI collection');
```

    - Chart

```
varNDVIchart = ui.Chart.image.series({
  imageCollection: healthycol,
  region: roi,
  reducer: ee.Reducer.mean(),
  scale: 30
});

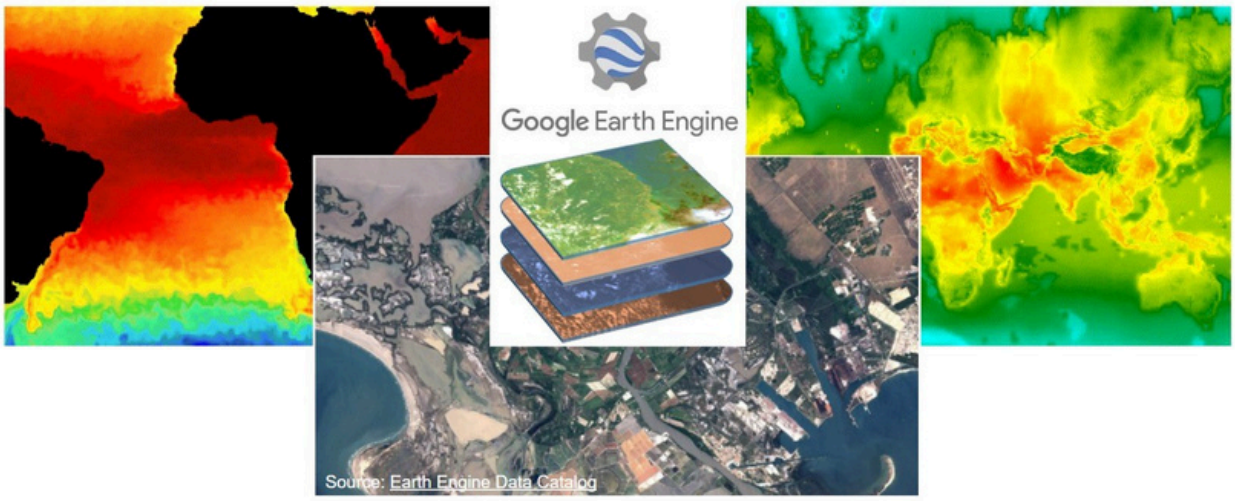
// Chart
print(NDVIchart);

// Set the options of the chart and print it
varoptions = {
```

```
  hAxis:  {  title:'Date'},
  vAxis:  {  title:'NDVI'},
  series: {
    0: { color:'gray'}
  }
};

//Uncomment this
NDVIchart.setOptions(options);
```

- All used commands are available in the Workshop-IPI22 repository part 1: *08_ImageStatistic&reduction*



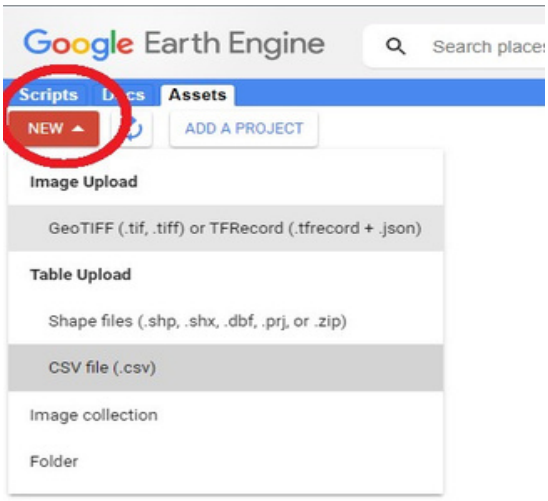
Introduction to Earth Engine

Image classification
Lab#5

5. Image Classification

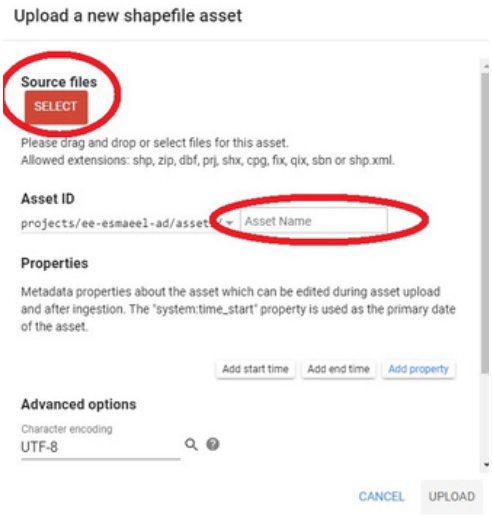
5.1 Preparing data

- Import Shapefile
  - ○GO to the assets tab in the left panel, click the "NEW" red button, and choose shapefile from the drop list

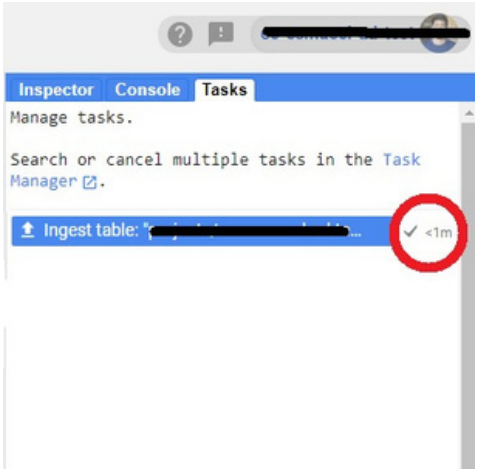


- Click select button and navigate to the location of the shp file at your drive and select it
  - Make sure the name of the file doesn't contain spaces
  - It is recommended that you put the files constituting the shp in one zipped folder

- Check the asset path where the file will be save

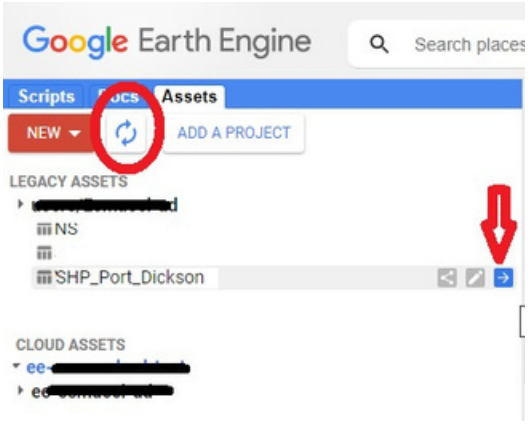


- Click upload and check the task tab in the right panel to monitor the uploading and ingesting processes



- Once the ingesting is complete you can find the file in the assets tab in the left panel (click refresh button if it doesn't show up automatically)

- Hover the pointer on the file name and click the small arrow which import the file directly to the imports section



- Rename the file in the imports section to “Port_Dickson”

- Convert the imported shapefile to a geometry and make a buffer of 1000m around it

```
varregion = Port_Dickson.geometry().buffer(1000);
Map.addLayer(region,{color: 'blue'}, "region", false);
Map.centerObject(region, 12);
```

- Prepare the image to be classified

  - In this example, we will work with the Sentinel-2 cloud free composite derived by applying the function from the previous lab
  - First copy paste the function provided in the script *06_CloudFreeComposite_Sentinel2*
  - Enter the function parameter to derive a cloud free composite over the previously imported shapefile and within specific time range taking the dry season for the year 2020

```
// The function should be pasted above//
// Get sentinel 2 cloud free image by calling the function and set
parameters
varCloudFreeS2 = GetS2CloudFree('2020-05-01','2020-09-01',40,
region);
// set vizualization for true color sentinel-2 image
varTColorS2Vis = {'bands': ['B4','B3','B2'],'min':0,'max':
3000};
// add the cloud free sentinel-2 image to the map
Map.addLayer(CloudFreeS2, TColorS2Vis ,'Cloud FreeS2');
```

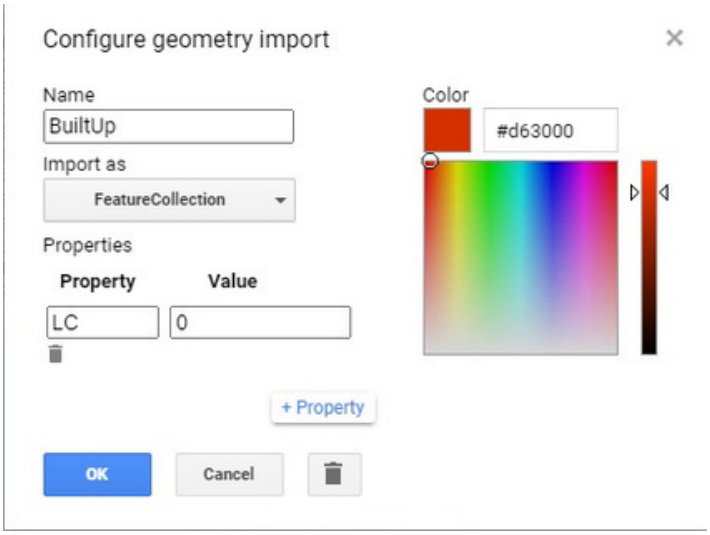
5.2 Creating training and validation data

- PreparingTraining dataandclassificationscheme (example)

    - Define five distinguishable Multipoints layers representing five classes: BuitUp, Water, DenseTree_Forests, Trees_plantation, OtherCrops
    - For each class, using the interactive drawing tool, crete ground truth points representing the land cover type samples of the class



    - Convert each layer to a feature collection and add class value properties using the interactive geometry import window
        - Name the property “LC” and use a sequence to populate the property value from 0 to 4 for the five classes



- Merge all the data into one layer and print the output to the console

```
//merge training data

varData =
BuiltUp.merge(Water).merge(DenseTree_Forests).merge(Trees_plantation)
.merge(OtherCrops);

print('Data&scheme:', Data);
```

- Now that we have the data ready. We need to split the data into training and validation data ; say 70% and 30 % respectively so we can use the training data to train the classifier and the validation data to check the accuracy metrics
- To do that we create a column with random values ranged from 0 to 1 and split based on the 70% by filtering the properties of this random column

```
// set random column to split
varrand = Data.randomColumn();

// 70% as training data
vartrainingD = rand.filter(ee.Filter.lt('random',0.7));

// 30% as validation data
varvalidationD = rand.filter(ee.Filter.gte('random',0.7));
```

- Finally, we will create a training sample that based on the training data and the corresponding bands values form the image we will classify.
  - To do that we first subset the image to the bands of interest by using select() as in previous labs

```
// select the relevant bands
varbands = ['B2','B3','B4','B5','B6','B7','B8','B8A',
'B11','B12', 'NDVI'];

varTimage = CloudFreeS2.select(bands);
```

  - To sample the image using the training points we use the sample.Regions

```
//create samples based on trainingD
vartrainingSamples = Timage.sampleRegions({
  collection: trainingD,
  properties: ['LC'],
  scale: 10
});
print('Training samples:', trainingSamples);
```

5.3 Classifying the image

- In this example, we will use the smile random forest classifier built-in algorithm . We can do by using the built-in function and feed it with the training samples, specifying the property we need to classify (the dependent variable) which is the land cover in our case “LC”, and input properties are the image bands (the independent variables)

```
//train the classifier
varclassifier = ee.Classifier.smileRandomForest(10).train({
  features: trainingSamples,
  classProperty: 'LC',
  inputProperties: bands
});
```

- We run the classifier using .classify() fed by the classifier we identify in the previous step

```
//run the classification
varclassified = Timage.classify(classifier);
```

- The result is the intended classified image that we need to add to the map
  - Notice min/max according to the scheme we choose (n. Of classes)

```
// Add the classified image to the map
Map.addLayer(classified,
{min:0, max:4, palette:
['7F5217', '1414FF','004E00', '509E50', 'D7D700']},
'classified');
```

- The previous script is available in *11_Classification3(Classify)*

5.4 Accuracy Assessment

- To calculatethe confusion matrix, error matrix, and accuracy metrics we use the built-in function for each parameter
- First from training data and the classifier, we can check the confusion matrix

```
//Confusion matrix from the training data
varconfusionMatrix = classifier.confusionMatrix();
print('Confusion matrix: ', confusionMatrix);
```

- To conduct the accuracy assessment on the validation data we use a validation sample, i.e same as the training sample but using the validation data subset

```
// sample the validation data
varvalidation = classified.sampleRegions({
  collection: validationD,
  properties: ['LC'],
  scale: 10,
});
print('Validation:', validation);
```

- Once we get the validation sample we can calculate the error matrix using the errorMatrix function to compare the validation (truth) and classification results; therefore we provide an input the name of the property in the validation data (“LC”), and the name of the property from the classified image (it’s “classification” by default)

```
//Compare validation(truth) and the classification result

varErrorMatix = validation.errorMatrix('LC','classification');

//Print the error matrix to the console
print('Error matrix: ', ErrorMatix);
```

- To calculate the accuracy statistics: the overall accuracy and kappa coefficient, we use the ready to use function for each

```
varOverAllAccuracy = ErrorMatix.accuracy();
print('Overall Accuracy: ', OverAllAccuracy);

varkappa = ErrorMatix.kappa();
print('Validation Kappa', kappa);
```

- Finally, we can export the classified image

```
// Export the image to drive
Export.image.toDrive({
image: classified,
description: 'classified',
scale: 10,
maxPixels :1e13,
region: region
});
```

- The previous script is available in *12_Classification4(AccuracyAssessment)*

5.6 Unsupervised classification

- Like supervised classification, we first need to prepare a clear image. For this example we will use Landsat cloud free composite derived as in *05_CloudFreeComposite_Landsat8.*The output of thisscript is a cloud free composite; “CompositeCloudFree”
- Unlike supervised classification, we don’t need training data to classify, but we still need to make a training sample to cluster the image

```
// Make the training dataset
vartraining = CompositeCloudFree.sample({
  region: Selangor,
  scale: 30,
  numPixels: 5000
});
```

- We use a cluster and Kmeans algorithm and specify the number of classes (choose 4 for this example).
-

```
// Instantiate the clusterer and train it
varclusterer = ee.Clusterer.wekaKMeans(4).train(training);
```

- Similar to what we did in the supervised classification, we apply the clusterrer using the cluster function fed by the clusterer we specified. The results is the classified that we can add to the map

```
// Cluster the input using the trained clusterer.
varresult = CompositeCloudFree.cluster(clusterer);

// Display the clusters with random colors.
Map.addLayer(result, {min:0, max:3, palette:['7F5217',
'004E00','1414FF', 'D7D700']}, 'clusters');
```

- The previous script is available in *13_UnsupervisedClassification_Cluster*